

Q1. Cosa si intende per "spazio metrico" e in che modo la "metrica euclidea su $\mathbb{R}$ " vi si relaziona?

Uno **spazio metrico** è una coppia (X, d) con X insieme non vuoto e $d : X \times X \rightarrow \mathbb{R}$ (distanza) che per ogni x, y, z soddisfa 4 assiomi:

1. **Non-negatività:** $d(x, y) \geq 0$
2. **Identità:** $d(x, y) = 0 \Leftrightarrow x = y$
3. **Simmetria:** $d(x, y) = d(y, x)$
4. **Disuguaglianza triangolare:** $d(x, z) \leq d(x, y) + d(y, z)$

La **metrica euclidea su $\mathbb{R}$** è $d(x, y) = |x - y|$: soddisfa i 4 assiomi (la triangolare segue da $|a + b| \leq |a| + |b|$) ed è il caso prototipo. Si generalizza a $\mathbb{R}^N$ con $d(x, y) = \|x - y\|_2 = \sqrt{\sum_i (x_i - y_i)^2}$, dove la triangolare segue da Cauchy-Schwarz. **Perché serve / collegamento:** limite, continuità, convergenza e ottimizzazione richiedono una nozione di "vicinanza". La discesa del gradiente vive in $(\mathbb{R}^N, \|\cdot\|_2)$ e un'immagine digitale è un punto di $\mathbb{R}^N$ con $N = larghezza \times altezza \times canali$.

Q2. Come si definisce la "continuità" per una funzione @@INL38@@, e qual è il significato di "continuità separata"?

f è **continua** in x_0 se:

$$\forall \varepsilon > 0, \exists \delta > 0 : \|x - x_0\| < \delta \Rightarrow |f(x) - f(x_0)| < \varepsilon$$

Cioè: a piccole variazioni dell'input corrispondono piccole variazioni dell'output (Cauchy generalizzata da intervalli a "palle" in $\mathbb{R}^N$). La **continuità separata** richiede che f sia continua rispetto a una variabile alla volta, tenendo fisse le altre. **Relazione:** la continuità (congiunta) implica quella separata, **ma non viceversa**. **Controesempio** (da sapere): $f(x, y) = xy/(x^2 + y^2)$ con $f(0, 0) = 0$. Sugli assi vale 0 (separatamente continua nell'origine), ma sulla retta $y = x$ vale $1/2$ per ogni $x \neq 0$: non ha limite in $(0, 0)$, quindi non è continua. **Collegamento:** è la prima prova che il calcolo in più variabili non è la semplice somma di calcoli a una variabile - serve controllare *ogni* direzione.

Q3. Qual è la relazione tra "derivata direzionale", "derivata parziale" e "gradiente" nel calcolo in più variabili?

- **Derivata direzionale** lungo il versore $\mathbf{v}$: $D_{\mathbf{v}}f(x_0) = \lim_{h \rightarrow 0} [f(x_0 + h\mathbf{v}) - f(x_0)]/h$. Tasso di variazione di $\mathbf{f}$ muovendosi lungo $\mathbf{v}$.
- **Derivata parziale**: derivata direzionale lungo un versore della base canonica $\mathbf{e}_i$ (deriva rispetto a x_i , fissate le altre).
- **Gradiente**: vettore delle derivate parziali $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$.

Relazione (se $\mathbf{f}$ differenziabile): $D_{\mathbf{v}}f = \nabla f \cdot \mathbf{v}$. Conseguenze:

- il gradiente contiene tutte le derivate direzionali;
- per Cauchy-Schwarz $|D_{\mathbf{v}}f| \leq \|\nabla f\|$, con uguaglianza se $\mathbf{v} \parallel \nabla f$: quindi ∇f **punta nella direzione di massima crescita** (e $-\nabla f$ di massima decrescita);
- $\nabla f = 0$ è un punto critico (minimo, massimo o sella).

Collegamento: è la proprietà di massima decrescita che fa funzionare la discesa del gradiente.

Q4. Descrivi il "metodo iterativo di discesa del gradiente" e la sua finalità.

Finalità: trovare un minimo di una funzione differenziabile f (es. la funzione di costo di una rete) quando non c'è soluzione in forma chiusa. **Idea:** poiché $-\nabla f$ è la direzione di massima decrescita locale, ci si muove a ogni passo in quella direzione. Intuizione: una pallina che rotola lungo la superficie scegliendo sempre la pendenza più ripida verso il basso. **Algoritmo:** scelto x_0 e un **learning rate** $\eta > 0$:

$$x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$$

fino a un criterio di arresto ($\|\nabla f\|$ piccola, max iterazioni). **Learning rate:** troppo grande $\rightarrow$ oscilla/diverge; troppo piccolo $\rightarrow$ lentissimo. **Garanzia:** converge al minimo globale solo se f è convessa; altrimenti a un punto critico. **Collegamento:** è il motore dell'addestramento delle reti. Varianti: Batch GD, **SGD** (un esempio per volta, rumoroso ma veloce), Mini-batch (standard nel DL).

Q5. Descrivi il metodo della discesa del gradiente. In quali casi potrebbe non convergere al minimo globale?

Metodo: $x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$ (vedi Q4): si sfrutta $-\nabla f$ come direzione di massima decrescita. Garantisce solo la convergenza a un **punto critico**. Non raggiunge il minimo globale quando:

1. **Funzione non convessa con minimi locali** (tipico delle reti, es. $f(x) = x^4 - 3x^2 + x$): si ferma nel primo minimo locale.
2. **Punti di sella** ($\nabla f = 0$ ma non minimo), prevalenti in alta dimensione.
3. **Plateau** con gradiente quasi nullo (es. sigmoidi sature $\rightarrow$ vanishing gradient): avanzamento bloccato.
4. **Learning rate sbagliato**: troppo grande $\rightarrow$ diverge; troppo piccolo $\rightarrow$ non arriva nel tempo utile.
5. **Inizializzazione sfortunata**: finisce nel bacino di un minimo locale scadente (da qui le init Xavier/He).

In sintesi: minimo globale garantito **solo se f è convessa**. Nel deep learning si accetta il limite perché i minimi locali trovati danno comunque buoni modelli.

Q6. Come funziona un Single Layer Perceptron (SLP)? Quali sono i suoi limiti?

L'SLP ha ingressi $\mathbf{x}$, pesi $\mathbf{w}$, bias b e attivazione φ (storicamente il gradino). Calcola:

$$z = w \cdot x + b, y = \varphi(z) \text{ con } \varphi_{\text{gradino}} : 1 \text{ se } z \geq 0, \text{ else } 0$$

Geometricamente definisce un **iperpiano** $w \cdot x + b = 0$ e classifica i punti in base al lato in cui cadono: è un **classificatore lineare binario**. Si addestra con la regola del perceptron, che converge se i dati sono linearmente separabili. **Limiti**:

1. Risolve **solo problemi linearmente separabili**. **Esempio classico**: non risolve lo **XOR** (Minsky-Papert), perché nessun iperpiano separa $\{(0,0), (1,1)\}$ da $\{(0,1), (1,0)\}$.
2. Una sola superficie di decisione lineare: niente funzioni non lineari.
3. Il gradino non è differenziabile $\rightarrow$ niente backpropagation.

Collegamento: si superano con il **MLP** (strati nascosti + attivazioni non lineari), che è un approssimatore universale (Q10).

Q7. Confronta le funzioni di attivazione ReLU, sigmoide e tanh in termini di proprietà matematiche e applicazioni.

	Formula	Codominio	Note
Sigmoide	$\sigma(z) = 1/(1 + e^{-z})$	$(0,1)$	satura $\rightarrow$ vanishing gradient; non centrata in 0
tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$(-1,1)$	centrata in 0; satura comunque
ReLU	$\max(0, z)$	$[0, \infty)$	no saturazione per $z > 0$; veloce; "dying ReLU" per $z < 0$

- **Sigmoide:** $\sigma' = \sigma(1 - \sigma) \leq 1/4$ (max in 0); interpretabile come probabilità. Uso: output binario, gate LSTM.
- **tanh:** $\tanh' = 1 - \tanh^2$; output centrato $\rightarrow$ gradienti più bilanciati della sigmoide. Uso: hidden layer RNN.
- **ReLU:** derivata 1 per $z > 0 \rightarrow$ niente vanishing gradient, calcolo banale, induce sparsità. È lo **standard** negli hidden layer delle reti deep.

Collegamento: la saturazione di sigmoide/tanh è la causa del vanishing gradient (Q31); ReLU è la principale ragione per cui si addestrano reti profonde.

Q8. Spiega il principio dell'algoritmo di backpropagation in un MLP con un layer nascosto.

MLP con un layer nascosto:

$$h = \varphi(W^{(1)}x + b^{(1)}), \hat{y} = \psi(W^{(2)}h + b^{(2)})$$

La **backpropagation** calcola i gradienti della loss rispetto a tutti i parametri applicando la **regola della catena** all'indietro. **Forward**: calcola e memorizza $z^{(1)}, h, z^{(2)}, \hat{y}$ e la loss L . **Backward** (propaga il "segnale di errore" δ):

$$\delta^{(2)} = \partial L / \partial \hat{y} \cdot \psi'(z^{(2)}) \partial L / \partial W^{(2)} = \delta^{(2)} h^T$$

$$\delta^{(1)} = (W^{(2)T} \delta^{(2)}) \odot \varphi'(z^{(1)}) \partial L / \partial W^{(1)} = \delta^{(1)} x^T$$

Update: $W \leftarrow W - \eta \cdot \partial L / \partial W$ (discesa del gradiente). **Perché conta**: riutilizzando i calcoli intermedi, il costo di un passo è $\sim 2\times$ il forward, **indipendentemente dalla profondità**. Senza, servirebbe stimare ogni derivata con differenze finite (un forward per parametro): impossibile con milioni di pesi. È ciò che ha sbloccato il deep learning.

Q9. Cos'è una funzione di costo e quale ruolo svolge nel processo di apprendimento? Fai un esempio con la MSE.

Una **funzione di costo** $L(\theta)$ misura quanto le predizioni del modello si discostano dai target sul training set. Ruoli: **definisce l'obiettivo** ($\theta^* = \operatorname{argmin}_{\theta} L(\theta)$), **guida l'ottimizzazione** ($\nabla_{\theta} L$ indica come aggiornare i parametri), **codifica il problema** (loss diverse per regressione, classificazione, modelli generativi). **Esempio - MSE** (regressione):

$$L(\theta) = (1/n) \sum_i (f_{\theta}(x_i) - t_i)^2$$

Proprietà: differenziabile (gradiente $2(\hat{y} - t)$, ideale per backprop); penalizza quadraticamente gli errori grandi (un errore doppio pesa $4\times \rightarrow$ sensibile agli outlier). **Collegamento:** minimizzare la MSE equivale alla **massima verosimiglianza** con rumore gaussiano; per la classificazione si usa invece la cross-entropy (Q29).

Q10. Qual è il significato del teorema di approssimazione universale per le reti neurali?

Una rete con **un solo layer nascosto** e attivazione non polinomiale, con abbastanza neuroni, può approssimare **qualunque funzione continua** su un compatto con precisione arbitraria:

$$F(x) = \sum_i \alpha_i \varphi(w_i \cdot x + b_i), \sup_K |F(x) - f(x)| < \varepsilon$$

Significato: gli MLP sono **approssimatori universali**: nessuna funzione continua è fuori portata. **Cosa NON dice:** quanti neuroni servono (può essere esponenziale nella dimensione - *curse of dimensionality*); come trovare i pesi (è un risultato di sola **esistenza**, non costruttivo); nulla sulla generalizzazione su dati nuovi. **Collegamento:** risponde all'obiezione post-SLP (Q6) sulla limitatezza delle reti; ma in pratica le reti **profonde** ottengono lo stesso con molti meno neuroni (vantaggio della profondità).

Q11. Quali sono i componenti fondamentali di un "perceptrone" o "neurone artificiale", e a cosa servono i "pesi" e il "bias"?

Componenti: **ingressi** x , **pesi** w , **bias** b , **attivazione** φ . Calcolo:

$$z = w \cdot x + b, y = \varphi(z)$$

- **Pesi:** misurano l'importanza di ogni input (w_i grande $\rightarrow$ input influente; $w_i < 0 \rightarrow$ relazione inversa). Geometricamente w è il **vettore normale** all'iperpiano di decisione $w \cdot x + b = 0$.
- **Bias:** **trasla** l'iperpiano; senza, passerebbe sempre per l'origine. **Esempio:** per attivarsi quando $x > 5$, con $b = -5w$ la condizione $wx - 5w \geq 0 \Leftrightarrow x \geq 5$ diventa rappresentabile. Rende la decisione **affine** invece che solo lineare.

Collegamento: spesso il bias si "assorbe" estendendo $x \leftarrow (1, x)$ e $w \leftarrow (b, w)$, riducendo tutto a un prodotto scalare. È il mattone base di MLP e CNN.

Q12. Definisci la "funzione di costo" e illustra la differenza tra "loss L1" (geometria del taxi) e "loss L2" (MSE).

Funzione di costo: $L(\theta)$ che misura l'errore medio del modello (vedi Q9).

- **L2 (MSE):** $\ell = (\hat{y} - t)^2$, per vettori $\|\hat{y} - t\|_2^2$. Distanza **euclidea**.
- **L1 (MAE):** $\ell = |\hat{y} - t|$, per vettori $\|\hat{y} - t\|_1$. Distanza **Manhattan** (taxi): come si muove un taxi lungo le strade di una griglia.

	L1 (MAE)	L2 (MSE)
Crescita errore	lineare	quadratica
Differenziabile in 0	no (cuspidi)	sì
Gradiente	± 1 costante	$2(\hat{y} - t)$
Outlier	robusta	sensibile
Stimatore ottimo	mediana	media
Interpretazione MLE	rumore Laplace	rumore Gauss

L2 amplifica gli errori grandi (buon segnale di convergenza ma sensibile agli outlier); L1 è robusta ma con gradiente costante (oscilla vicino al minimo). **Compromesso: Huber loss** (L2 per errori piccoli, L1 per quelli grandi).

Q13. Elenca e descrivi brevemente almeno quattro "funzioni di attivazione" comunemente utilizzate nelle reti neurali artificiali.

L'attivazione φ introduce **non linearità**: senza, una rete profonda collasserebbe in una sola trasformazione lineare.

1. **Sigmoide** $\sigma(z) = 1/(1 + e^{-z}) \in (0,1)$: interpretabile come probabilità; satura (vanishing gradient).
Uso: output binario, gate.
2. **tanh** $\in (-1,1)$: come la sigmoide ma centrata in 0 $\rightarrow$ gradienti più bilanciati. Uso: hidden layer RNN.
3. **ReLU** $\max(0, z)$: nessuna saturazione per $z > 0$, calcolo banale, sparsità. Standard negli hidden layer; difetto: dying ReLU.
4. **Softmax** $e^{z_i} / \sum_j e^{z_j}$: trasforma un vettore in distribuzione di probabilità (somma 1). Uso: output classificazione multiclasse.

Collegamento: la scelta dipende dal ruolo del layer (ReLU nei nascosti, sigmoide/softmax in output).
Varianti moderne: Leaky ReLU, GELU.

Q14. Qual è il principio su cui si basa l'algoritmo di "backpropagation" e come mai è importante per l'addestramento di MLP?

Si basa su due idee:

1. **Regola della catena:** una rete è una **composizione** di funzioni $f_L \circ \dots \circ f_1$, quindi il gradiente è il prodotto delle derivate locali lungo gli strati.
2. **Calcolo all'indietro** (reverse-mode autodiff): si propaga il "segnale di errore" dall'output all'input riutilizzando i calcoli intermedi:

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot \varphi'(z^{(l)})$$

da cui direttamente $\partial L / \partial W^{(l)}$ e $\partial L / \partial b^{(l)}$. **Importanza:** calcola **tutti** i gradienti in tempo $O(P)$, come un solo forward pass (con differenze finite servirebbero P forward $\rightarrow$ training impossibile). È **generale** (qualunque grafo differenziabile: MLP, CNN, RNN) e **modulare** (ogni layer espone forward+backward). Ha reso possibile il deep learning (Rumelhart-Hinton-Williams, 1986).

Q15. Cosa afferma il "Teorema di approssimazione universale"?

Afferma che una rete feedforward con **un solo layer nascosto** e attivazione non polinomiale (sigmoide, tanh, ReLU...), con un numero sufficiente di neuroni, può approssimare **qualsunque funzione continua su un compatto** con precisione arbitraria:

$$F(x) = \sum_i \alpha_i \varphi(w_i \cdot x + b_i), \sup_K |F(x) - f(x)| < \varepsilon$$

Le combinazioni di questa forma sono **dense** nello spazio $C(K)$ delle funzioni continue (Cybenko 1989; Hornik 1991 lo estende a ogni attivazione non polinomiale). È la **giustificazione teorica** delle reti neurali come modelli generali. **Limiti:** non dice quanti neuroni servano, né che la discesa del gradiente trovi i pesi buoni, né garantisce la generalizzazione. (Stesso contenuto di Q10.)

Q16. Descrivere le fasi principali del processo di "digitalizzazione delle immagini", specificando i concetti di "campionamento" e "quantizzazione".

La digitalizzazione converte un'immagine continua $f : \mathbb{R}^2 \rightarrow \mathbb{R}^+$ in una matrice di pixel, in due fasi principali:

- **Campionamento:** discretizzazione del **dominio spaziale**. Si valuta $\mathbf{f}$ su una griglia $M \times N$ di punti (i pixel) $\rightarrow$ definisce la **risoluzione**. Campionare troppo poco causa **aliasing** (moiré, bordi a scalini); il **teorema di Nyquist** richiede frequenza di campionamento $\geq$ doppio della massima frequenza del segnale (si filtra passa-basso prima).
- **Quantizzazione:** discretizzazione del **codominio** (intensità). I valori continui vengono mappati su L livelli (tipicamente $L = 256$, 8 bit). Pochi livelli $\rightarrow$ **banding/posterizzazione**.

Risultato: immagine digitale $I \in 0, \dots, L-1^{M \times N \times C}$ ($C = 1$ grigi, $C = 3$ RGB). **Trade-off:** più fini campionamento e quantizzazione $\rightarrow$ più fedeltà ma più memoria.

Q17. Cosa si intende per immagine digitale? Spiega i concetti di campionamento, quantizzazione e spazio colorimetrico.

Un'immagine digitale è una matrice (tensore) di pixel $H \times W \times C$, ottenuta per digitalizzazione.

- **Campionamento:** discretizzazione spaziale su griglia $\rightarrow$ risoluzione (vedi Q16). Es. una foto $1920 \times 1080 \approx 2$ milioni di pixel.
- **Quantizzazione:** discretizzazione dei valori su L livelli (8 bit $\rightarrow$ 256, 0=nero, 255=bianco).
- **Spazio colorimetrico:** il sistema con cui i numeri del pixel diventano colore percepito:
- **RGB:** additivo, 3 canali, standard per i display;
- **Scala di grigi:** 1 canale, $Y = 0.299R + 0.587G + 0.114B$ (pesi = sensibilità percettiva verde>rosso>blu);
- **HSV:** tinta/saturazione/valore, utile per selezioni semantiche per colore;
- **YCbCr:** separa luminanza e cromaticanza, sfruttato nella compressione (JPEG sotto-campiona la cromaticanza).

Collegamento: la conversione in luminanza è il primo passo del progetto Canny.

Q18. Come si utilizza un istogramma per operazioni di elaborazione su immagini in scala di grigi?

L'**istogramma** $h(k)$ conta i pixel con intensità k ; normalizzato $p(k) = h(k)/(MN)$ è la distribuzione delle intensità. Rivela esposizione (spostato a sinistra/destra = sotto/sovra-esposto) e contrasto (stretto = basso contrasto). Operazioni:

- **Stretching del contrasto:** rimappa linearmente il range usato $[a, b]$ su $[0, L - 1]$.
- **Equalizzazione:** usa la CDF come trasformazione, $I' = (L-1) \cdot CDF(I) \rightarrow$ istogramma $\sim$ uniforme, massimizza il contrasto globale.
- **Sogliatura (thresholding):** scelta una soglia T (anche automatica, **Otsu**, che massimizza la varianza inter-classe) si binarizza per separare oggetto/sfondo; un istogramma **bimodale** aiuta a scegliere T nella "valle".

Limite/collegamento: l'istogramma **ignora la struttura spaziale** (due immagini diverse possono avere lo stesso istogramma); per il contesto locale servono i filtri (es. Sobel, Q19).

Q19. Spiega come funziona il filtro di Sobel e quale informazione estrae da un'immagine.

Il **Sobel** approssima il **gradiente** dell'intensità per estrarre i **bordi** (luoghi di forte variazione di luminosità). Usa due kernel 3×3 :

$$S_x = \begin{vmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{vmatrix} \quad S_y = \begin{vmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{vmatrix}$$

Per convoluzione $G_x = I * S_x$, $G_y = I * S_y$, da cui:

$$M = \sqrt{G_x^2 + G_y^2}(\text{magnitudo} : \text{forzadelbordo})$$

$$\theta = \arctan2(G_y, G_x)(\text{direzionedelbordo})$$

M alta = bordo, bassa = zona uniforme; sogliaando M si ottiene la mappa dei bordi. **Perché funziona:** un bordo è una transizione rapida di intensità, quindi una derivata grande. I pesi (1,2,1) fanno anche uno **smoothing** gaussiano implicito $\rightarrow$ meno sensibile al rumore del Prewitt (pesi uniformi).

Collegamento: è il secondo stadio della pipeline di **Canny** (progetto d'esame), dopo lo smoothing gaussiano e prima del non-maximum suppression.

Q20. Qual è la differenza tra convoluzione e cross-correlazione? Fornisci una definizione formale.

Cross-correlazione (2D): il kernel scorre e si applica direttamente:

$$(I \star K)[i, j] = \sum_{u, v} I[i + u, j + v] \cdot K[u, v]$$

Convoluzione (2D): come sopra ma con il **kernel ribaltato** (segno meno, cioè riflesso di 180°):

$$(I * K)[i, j] = \sum_{u, v} I[i - u, j - v] \cdot K[u, v]$$

Differenza: per kernel **simmetrici** (gaussiano) coincidono; per kernel **asimmetrici** (Sobel) differiscono per uno specchio/segno. Solo la **convoluzione** è commutativa, associativa e soddisfa il teorema di Fourier. **Collegamento:** le CNN, nonostante il nome, usano la **cross-correlazione**: i pesi sono appresi, quindi il flip è irrilevante. Nei sistemi LTI e nell'immagine processing classico (Canny) la convoluzione "vera" è invece obbligatoria.

Q21. Quali sono le proprietà principali dell'operazione di convoluzione?

1. **Commutativa:** $f * g = g * f$
2. **Associativa:** $(f * g) * h = f * (g * h) \rightarrow$ permette di **comporre filtri** in un unico filtro $g * h$
3. **Distributiva** sulla somma: $f * (g + h) = f * g + f * h$
4. **Lineare:** $(\alpha f) * g = \alpha(f * g)$
5. **Elemento neutro:** la delta di Dirac, $f * \delta = f$
6. **Differenziazione:** $(f * g)' = f' * g = f * g' \rightarrow$ es. $(I * G)' = I * G'$ (convolvo con la derivata della gaussiana, evitando di derivare l'immagine)
7. **Teorema di convoluzione:** $Ff * g = Ff \cdot Fg \rightarrow$ in frequenza diventa prodotto, calcolo via FFT in $O(N \log N)$
8. **Invarianza per traslazione** $\rightarrow$ base dell'**equivarianza** delle CNN
9. **Separabilità:** se $K = Kx \cdot Ky$, costo da $O(k^2)$ a $O(k)$ (es. gaussiana, Sobel)

Collegamento: separabilità + FFT rendono i filtri efficienti; l'invarianza per traslazione è ciò che le CNN sfruttano.

Q22. Illustra l'utilizzo dell'"istogramma dei livelli di grigio" in relazione agli "operatori puntuali" per la manipolazione di immagini luminanza.

Un **operatore puntuale** trasforma ogni pixel in funzione del solo suo valore: $I'[i, j] = T(I[i, j])$, **senza guardare i vicini**. L'effetto sull'istogramma è prevedibile: se T è iniettiva $h'(T(k)) = h(k)$, altrimenti $h'(j) = \sum_{T(k)=j} h(k)$. L'istogramma è quindi sia **diagnostico** (esposizione, contrasto) sia **dato di lavoro** per progettare T . Principali operatori:

- **Negativo:** $T(k) = (L - 1) - k$
- **Correzione gamma:** $T(k) = (L - 1)(k/(L - 1))^\gamma$ ($\gamma < 1$ schiarisce le ombre)
- **Contrast stretching:** $T(k) = (L - 1)(k - a)/(b - a)$
- **Equalizzazione:** $T(k) = (L - 1) \cdot CDF(k) \rightarrow$ istogramma uniforme
- **Sogliatura:** $T(k) = 0$ se $k < T$, altrimenti $L - 1$

Si implementano via **lookup table** ($O(1)$ per pixel, parallelizzabili). **Limite:** ciechi al contesto spaziale (non riducono rumore, non rilevano bordi).

Q23. In che modo i "polinomi di Taylor" vengono applicati per la creazione di "filtri di sharpening" come il filtro di Sobel o il filtro Laplaciano?

Le derivate discrete di un'immagine si ricavano dallo **sviluppo di Taylor** dei pixel vicini. **Derivata prima** (Sobel): sottraendo gli sviluppi di $I[i, j + 1]$ e $I[i, j - 1]$:

$$\partial I / \partial x \approx (I[i, j + 1] - I[i, j - 1]) / 2 \rightarrow \text{kernel}[-1, 0, +1]$$

mediato sulle tre righe con pesi (1,2,1) dà il kernel di Sobel (derivata + smoothing). **Derivata seconda** (Laplaciano): sommando gli sviluppi:

$$\partial^2 I / \partial x^2 \approx I[i, j + 1] - 2I[i, j] + I[i, j - 1]$$

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2 \rightarrow \text{kernel}[[0, 1, 0], [1, -4, 1], [0, 1, 0]]$$

Sharpening: $I_{\text{sharp}} = I - k \cdot \nabla^2 I$ (il $-$ perché $\nabla^2 I$ è negativo nei picchi): accentua i bordi rendendo l'immagine più nitida. **In sintesi:** i filtri differenziali sono **polinomi di Taylor discretizzati** sui vicini di un pixel.

Q24. Definisci l'operazione di convoluzione nel continuo e indicane le principali proprietà, sia in una che in due variabili.

1 variabile:

$$(f * g)(t) = \int f(\tau)g(t - \tau)d\tau$$

2 variabili:

$$(f * g)(x, y) = \iint f(u, v)g(x - u, y - v)dudv$$

Concettualmente: si fa scorrere **g** (riflessa) su **f** e si integra il prodotto: per ogni posizione è una **media pesata** di **f** con pesi **g**. Proprietà: commutativa, associativa, distributiva, lineare; elemento neutro δ ; differenziazione $(f * g)' = f' * g$; teorema di convoluzione (Fourier $\rightarrow$ prodotto); invarianza per traslazione; separabilità in 2D (la gaussiana 2D è prodotto di due gaussiane 1D). (Dettagli in Q21.)

Collegamento: è un'operazione **lineare e invariante per traslazione** (LTI), linguaggio comune di filtri immagine, sistemi fisici (risposta impulsiva) e CNN.

Q25. Quali sono le differenze strutturali e funzionali tra un MLP e una CNN?

Aspetto	MLP	CNN
Connettività	fully-connected	locale (kernel)
Pesi	indipendenti	condivisi
N. parametri	altissimo	ridotto
Input	vettore 1D	tensore $H \times W \times C$
Equivarianza traslazione	no	sì

L'MLP appiattisce l'immagine perdendo la struttura spaziale; la CNN preserva la griglia 2D ed estrae feature locali con pesi condivisi. **Esempio quantitativo:** su un'immagine $224 \times 224 \times 3$, il primo strato di un MLP con 1000 unità ha $\sim 1.5 \cdot 10^8$ parametri; un conv $3 \times 3 \times 3 \rightarrow 64$ ne ha 1728 (5 ordini di grandezza in meno). **In sintesi:** una CNN è un MLP con vincoli (località + condivisione pesi) che incorporano l'**inductive bias** delle immagini $\rightarrow$ molti meno parametri ed equivarianza per traslazione.

Q26. Perché le "Convolutional Neural Networks (CNN)" sono particolarmente efficaci per la gestione di "dati a griglia", come le immagini?

Sfruttano per costruzione tre proprietà dei dati a griglia (che un MLP dovrebbe scoprire da zero):

1. **Località:** i pattern (bordi, texture) sono in regioni locali $\rightarrow$ kernel piccoli; strati profondi vedono regioni più grandi (campo recettivo crescente).
2. **Condivisione dei pesi:** lo stesso kernel ovunque $\rightarrow$ **equivarianza per traslazione** (un occhio è riconosciuto in ogni posizione), pochi parametri (un kernel 3×3 ha $9 \cdot C \cdot C'$ pesi indipendentemente dalla risoluzione).
3. **Composizionalità gerarchica:** con conv + pooling si costruiscono feature sempre più astratte (bordi $\rightarrow$ parti $\rightarrow$ oggetti), rispecchiando come gli oggetti sono fatti di parti.

In più il **pooling** dà invarianza a piccole traslazioni/deformazioni. **In sintesi:** l'efficacia nasce dall'allineamento tra architettura e **simmetrie naturali** del dato; imporle come inductive bias (invece di apprenderle) riduce i parametri e migliora la generalizzazione.

Q27. Dare una breve descrizione dei minimi esempi di strati utilizzati in una CNN: "layer convoluzionale", "funzione di attivazione ReLU", "layer di pooling", e "dropout".

- **Convoluzionale:** applica K filtri appresi (es. 3×3) $\rightarrow$ feature map. $y[i, j, k] = \sum W_k[u, v, c] \cdot x[i + u, j + v, c] + b_k$. Estrae feature locali. Iperparametri: kernel size, stride, padding, n. filtri.
- **ReLU:** $\max(0, z)$ element-wise dopo la conv. Introduce non linearità, niente vanishing gradient per $z > 0$, induce sparsità.
- **Pooling** (es. $\max 2 \times 2$, stride 2): aggrega vicini $\rightarrow$ riduce la risoluzione (meno calcolo), dà **invarianza a piccole traslazioni**, aumenta il campo recettivo. Senza parametri.
- **Dropout:** in training spegne ogni neurone con probabilità p (e riscalda di $1/(1 - p)$ per mantenere l'aspettativa). **Regolarizzazione** anti-overfitting (effetto ensemble: allena implicitamente tante sotto-reti). Disattivato in inference.

Stack tipico: $Conv \rightarrow ReLU \rightarrow Pool \rightarrow \dots \rightarrow Flatten \rightarrow FC \rightarrow Dropout \rightarrow FC \rightarrow Softmax$.
(Aggiunte comuni: BatchNorm, skip/residual.)

Q28. Per quale tipo di dati sono concepite le "Recurrent Neural Networks (RNN)" e quale problematica affrontano?

Le **RNN** sono concepite per **dati sequenziali** (testo, audio, serie temporali, video) in cui l'**ordine** conta e ogni elemento dipende dal contesto precedente. Risolvono due limiti delle reti feedforward sulle sequenze: input di **lunghezza variabile** e **assenza di memoria**. Introducono uno **stato nascosto** aggiornato a ogni passo (pesi condivisi nel tempo):

$$h_t = \varphi(W_h h_{t-1} + W_x x_t + b)$$

h_t riassume tutto ciò che la rete "ricorda" di $(x_1, \dots, x_t)$. Si addestrano con la **backpropagation through time** (rete srotolata nel tempo). **Problema principale: vanishing/exploding gradient** su lunghe sequenze (il gradiente attraversa T moltiplicazioni per W_h) $\rightarrow$ non imparano dipendenze a lungo termine. **Soluzione: LSTM/GRU**, che con dei **gate** (forget/input/output) mantengono selettivamente la memoria in una cell state. (Oggi spesso soppiantate dai Transformer.)

Q29. Quali sono le principali caratteristiche della "funzione di costo cross-entropy" e della "funzione di attivazione softmax", e in quale contesto sono spesso utilizzate?

Coppia standard per la **classificazione multiclasse**. **Softmax** trasforma i logit $\mathbf{z}$ in distribuzione di probabilità:

$$\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}, \sum_i = 1$$

Generalizza la sigmoide a K classi; invariante per costante additiva (si sottrae $\max(z)$ per stabilità numerica). **Cross-entropy** misura la distanza dalla distribuzione vera; con target one-hot:

$$L = - \sum_i t_i \log p_i = - \log p_c (c = \text{classe vera})$$

È la **negative log-likelihood** (= massima verosimiglianza); la differenza con l'entropia di $\mathbf{t}$ è la divergenza di Kullback-Leibler. **Proprietà chiave:** insieme danno un **gradiente pulito** $\partial L / \partial z_i = p_i - t_i$ (differenza *predetto - vero*), che rende la backprop efficiente e stabile. **Uso:** classificazione multiclasse, modelli di linguaggio (next token). Per $K = 2 \rightarrow$ binary cross-entropy con sigmoide.

Q30. Descrivi l'approccio dei modelli "Generative Adversarial Networks (GAN)".

Le **GAN** (Goodfellow 2014) sono modelli generativi basati su due reti in **competizione** (non massimizzano una verosimiglianza esplicita, imparano per contraddittorio):

- **Generatore G**: da rumore z genera dati falsi $G(z)$, vuole ingannare D.
- **Discriminatore D**: stima la probabilità che un campione sia reale.

Si addestrano in un **gioco minimax**:

$$\min_G \max_D E_{x \sim \text{dati}} [\log D(x)] + E_z [\log(1 - D(G(z)))]$$

D massimizza (distinguere vero/falso), G minimizza (campioni credibili). All'equilibrio (Nash) $p_G = p_{\text{data}}$ e $D = 1/2$: D non distingue più. **Difficoltà**: addestramento **instabile**; **mode collapse** (G genera pochi campioni ripetitivi); vanishing gradient per G se D è troppo bravo $\rightarrow$ si usa la **non-saturating loss** $-\log D(G(z))$. **Collegamento**: oggi largamente sostituite dai **modelli a diffusione** per la generazione di immagini, ma restano competitive su volti ad alta risoluzione.

Q31. Quali problemi possono sorgere nella retropropagazione in MLP profondi (vanishing/exploding gradients)?

Nella backprop il gradiente di uno strato profondo è il **prodotto** di molte Jacobiane $J^{(k)} = W^{(k)T} \text{diag}(\varphi'(z^{(k)}))$. Se la norma di questa catena cresce o decresce esponenzialmente con la profondità si hanno due patologie speculari:

- **Vanishing:** $\|\partial L/\partial W\| \rightarrow 0$, gli strati iniziali non imparano. Cause: attivazioni saturanti (sigmoide/tanh, $\varphi' \approx 0$), pesi piccoli (Jacobiane con autovalori < 1).
- **Exploding:** $\|\partial L/\partial W\| \rightarrow \infty$, aggiornamenti enormi $\rightarrow$ divergenza/NaN. Cause: pesi grandi (autovalori > 1), RNN su lunghe sequenze.

Soluzioni: inizializzazione **Xavier/He** (varianza costante tra layer); attivazioni non saturanti (**ReLU**); **Batch Normalization**; **skip/residual connections** (ResNet, il gradiente bypassa le Jacobiane piccole $\rightarrow$ reti di centinaia di layer); **gradient clipping** (per l'exploding); gate (LSTM); optimizer adattivi (Adam). **In sintesi:** il deep learning moderno è in gran parte l'insieme di queste tecniche che stabilizzano la moltiplicazione di Jacobiane.

Q32. Spiega il funzionamento di una CNN, indicando il ruolo di ciascun tipo di layer (convoluzionale, pooling, lineare).

Una CNN trasforma un'immagine $H \times W \times C$ in un output (es. classi) tramite una **gerarchia di feature**: bordi $\rightarrow$ parti $\rightarrow$ oggetti. Ruolo dei layer:

- **Convoluzionale**: estrae **feature locali** con filtri condivisi ($y = \sum W_k \cdot x + b_k$). Località + condivisione pesi $\rightarrow$ equivarianza per traslazione e pochi parametri. Seguito da ReLU (non linearità).
- **Pooling**: **down-sampling** spaziale; riduce calcolo, dà invarianza a piccole traslazioni, aumenta il campo recettivo. Senza parametri.
- **Lineare (fully-connected)**: alla fine, combina **globalmente** le feature per la **classificazione** (+ softmax). Sta in fondo perché ha molti parametri e lavora su feature già compatte e semanticamente ricche.

Struttura tipica:

Input $\rightarrow$ [Conv $\rightarrow$ ReLU $\rightarrow$ Pool] $\times N \rightarrow$ Flatten $\rightarrow$ FC $\rightarrow$ Softmax

Collegamento: strati bassi = feature semplici (bordi), strati alti = feature complesse (oggetti). La sinergia conv+pool+FC, con l'inductive bias di località/condivisione, spiega il successo delle CNN sulle immagini. *Fine delle 32 domande.*